



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de 
HONORIS UNITED UNIVERSITIES

École Marocaine des Sciences de l'Ingénieur – EMSI Tanger

Filière : Ingénierie Informatique et Réseaux

Niveau : 1^{ère} Année Cycle Ingénieur

Cahier des Charges Fonctionnel
Projets de Programmation Orientée Objet – C++

Matière : Programmation Orientée Objet – C++

Enseignant : Pr. Youssef JDIDOU

Année universitaire : 2025–2026

Version provisoire — généré le 4 novembre 2025

Table des matières

1	Introduction	2
2	Barème global d'évaluation	2
3	Projet 1 – Gestion de flotte automobile connectée (3 membres)	3
4	Projet 2 – Mini-système bancaire sécurisé (4 membres)	4
5	Projet 3 – Gestion d'un établissement scolaire (5 membres)	5
6	Projet 4 – Réservation d'hôtel (3–4 membres)	6
7	Projet 5 – Bibliothèque multimédia (4–5 membres)	7

1 Introduction

Ce document propose cinq projets **C⁺⁺ (POO)** pensés pour des groupes de **3 à 5 étudiants**, alignés avec le syllabus : classes/objets, constructeurs/destructeurs, encapsulation, héritage (public/protected/private), polymorphisme, surcharge (fonctions/opérateurs), exceptions, collections (**vector/list**) et gestion de fichiers. Les projets sont *console-based* (sans interface graphique) afin de focaliser l'effort sur la conception objet et la qualité du code.

Pour chaque projet : objectifs, fonctionnalités, aspects techniques, jalons et critères d'évaluation sont fournis.

2 Barème global d'évaluation

Critère	Pondération
Fonctionnalités & respect du cahier des charges	30%
Maîtrise des concepts POO C ⁺⁺ (héritage, polymorphisme, exceptions, surcharge)	25%
Structure, lisibilité & documentation du code	20%
Travail d'équipe & gestion de projet (rôles, régularité)	15%
Soutenance finale & rapport	10%

Jalons communs (tous projets)

1. **Prototype (Novembre 2025)** : squelette des classes, menu principal, compilable.
2. **Version Alpha (Décembre 2025)** : coeur fonctionnel, gestion des fichiers, premiers tests.
3. **Version Finale (Janvier 2026)** : robustesse (exceptions), nettoyage, documentation.
4. **Soutenance (fin Janvier 2026)** : démo, rapport 3–5 pages, répartition claire des contributions.

3 Projet 1 – Gestion de flotte automobile connectée (3 membres)

Objectif général

Développer une application **console** gérant un parc de véhicules (voitures, camions, motos) : inventaire, entretiens, statistiques de consommation.

Fonctionnalités (minimum)

- Ajouter / modifier / supprimer un véhicule, rechercher par matricule.
- Planifier l'entretien en fonction du kilométrage et du type.
- Calculer la consommation moyenne du parc et le coût total.
- Exporter un rapport synthétique (.txt) de l'état du parc.

Aspects techniques (POO)

- Classes : `Vehicule` (base), `Voiture`, `Camion`, `Moto`.
- **Héritage simple** (public), attributs protégés, getters/setters.
- **Surcharge** des opérateurs `==` (matricule) et `«` (affichage).
- Collections `vector<shared_ptr<Vehicule>`, exceptions (entrées invalides).
- Fichiers : lecture/écriture état du parc.

Planning & livrables

Période	Livrable
Nov. 2025	Prototype : classes de base + menu CRUD minimal.
Déc. 2025	Alpha : entretiens, statistiques, fichiers.
Jan. 2026	Finale : robustesse, exceptions, documentation.

Répartition type (3 étudiants)

- **E1** : Modélisation des classes et logique métier (CRUD).
- **E2** : Statistiques, entretiens, exceptions.
- **E3** : Persistance fichiers, intégration, tests, rapport.

4 Projet 2 – Mini-système bancaire sécurisé (4 membres)

Objectif général

Simuler la création et la gestion de comptes (courant, épargne) avec opérations et historique.

Fonctionnalités (minimum)

- Créer des comptes, authentification simple (identifiant + mot de passe).
- Opérations : dépôt, retrait, virement inter-comptes.
- Calcul des intérêts (compte épargne), historique des transactions.
- Sauvegarde/chargement des opérations dans des fichiers.

Aspects techniques (POO)

- Compte (base), `CompteCourant`, `CompteEpargne`.
- Polymorphisme : `virtual afficher()`, `virtual appliquerInterets()`.
- Exceptions personnalisées : solde insuffisant, compte introuvable.
- Surcharge : « (affichage), + (fusion de comptes, optionnel).

Planning & livrables

Période	Livrable
Nov. 2025	Prototype : classes de base + sécurité minimale (login).
Déc. 2025	Alpha : opérations complètes + historique.
Jan. 2026	Finale : exceptions, fichiers, nettoyage.

Répartition type (4 étudiants)

- **E1** : Comptes & opérations de base.
- **E2** : Authentification & sécurité simple.
- **E3** : Historique & fichiers.
- **E4** : Tests, intégration, documentation.

5 Projet 3 – Gestion d’un établissement scolaire (5 membres)

Objectif général

Gérer des personnes (étudiants, professeurs), des cours et des notes, avec calculs de moyennes et exports.

Fonctionnalités (minimum)

- Gestion `Personne` → `Etudiant/Professeur`.
- Affectation des matières, saisie des notes, moyennes par matière et globale.
- Recherches (par nom, par matière), export des relevés (`.txt`).

Aspects techniques (POO)

- Héritage simple, encapsulation stricte (getters/setters), `vector`.
- Polymorphisme : `virtual afficher()` et appel dynamique.
- Surcharge : `==` (comparaison `Personne`), `<` (affichage).
- Smart pointers recommandés : `shared_ptr`.

Planning & livrables

Période	Livrable
Nov. 2025	Prototype : classes de base + menus.
Déc. 2025	Alpha : gestion matières/notes + calculs.
Jan. 2026	Finale : recherches, exports, robustesse.

Répartition type (5 étudiants)

- **E1** : Modèle `Personne`/`Etudiant`/`Professeur`.
- **E2** : Matières, saisie de notes, calculs.
- **E3** : Recherche & tri, affichage.
- **E4** : Persistance (fichiers), export.
- **E5** : Intégration, tests et rapport.

6 Projet 4 – Réservation d’hôtel (3–4 membres)

Objectif général

Gérer des clients, des chambres et des réservations avec calcul de coût de séjour et contrôle de disponibilité.

Fonctionnalités (minimum)

- CRUD clients & chambres, création/annulation de réservations.
- Calcul du coût et liste des chambres disponibles.
- Gestion des erreurs (chambre occupée, client inexistant).

Aspects techniques (POO)

- Classes : `Client`, `Chambre`, `Reservation`, `Hotel`.
- Héritage simple (si déclinaisons de chambres), surcharge `==`, `<<`.
- Smart pointers (`shared_ptr`) et exceptions.

Planning & livrables

Période	Livrable
Nov. 2025	Prototype : modèles + disponibilité basique.
Déc. 2025	Alpha : réservations & calculs.
Jan. 2026	Finale : exceptions, persistance, nettoyage.

Répartition type (3–4 étudiants)

- **E1** : Modèles & logique de réservation.
- **E2** : Calculs, listes, exceptions.
- **E3** : Fichiers, intégration, tests.
- **E4** (si 4) : Documentation & qualité.

7 Projet 5 – Bibliothèque multimédia (4–5 membres)

Objectif général

Gérer des médias (livres, vidéos, audios), rechercher, emprunter/retourner, et produire des statistiques.

Fonctionnalités (minimum)

- CRUD médias, recherche par titre/type, tri.
- Emprunts/retours, contrôles de disponibilité.
- Durée totale/moyenne pour les contenus audiovisuels.
- Export de rapports (.txt).

Aspects techniques (POO)

- **Media** (base), **Livre**, **Audio**, **Video**; option : **LivreAudio** (héritage multiple).
- Polymorphisme pour l’affichage et le calcul des statistiques.
- Surcharge «, ==; smart pointers (**unique_ptr** / **shared_ptr**).

Planning & livrables

Période	Livrable
Nov. 2025	Prototype : base de classes + menus.
Déc. 2025	Alpha : fonctionnalités principales + fichiers.
Jan. 2026	Finale : statistiques, robustesse, documentation.

Répartition type (4–5 étudiants)

- **E1** : Modèle de base & héritage.
- **E2** : Recherche, tri, affichage.
- **E3** : Persistance & emprunts/retours.
- **E4** : Statistiques & exceptions.
- **E5** (si 5) : Intégration, tests & rapport.